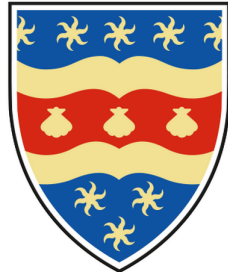


UNIVERSITY OF PLYMOUTH



BSc MATHEMATICS WITH THEORETICAL PHYSICS

MATH3713

Scaling Wind Farm Layout Optimisation to Meet UK Energy Demands

Author:

Grace RICHARDSON

Supervisor:

Dr. Matthew CRAVEN

Student ID

10745319

April 2025

Abstract

This report aims to explore the Wind Farm Layout Optimisation Problem (WFLOP) through both single- and multi-objective optimisation, to show that it can be used to create farm layouts that provide enough power for all homes in the UK. We highlight the intricacies of the problem, including the application of different algorithms and models, and the challenges of scaling the models to achieve optimal results. We employ both a Hill Climbing Algorithm (HCA) and an evolutionary algorithm (NSGA-II) to obtain wind farm layouts, and compare these to the well-known results proposed by Mosetti et al. Moreover, we present a novel multi-objective optimisation that aims to minimise the farm size whilst maximising power, with scaling farm boundaries and number of turbines. We conclude that the layout optimisation can be scaled to meet UK demands. However, further improvement to the calculations presented in this report would be required.

Acknowledgements

I would like to express my heartfelt gratitude to my supervisor, Matthew Craven, for his invaluable guidance and support throughout this journey. A special thanks to Pawel Manikowski, whose help with the Python code was crucial to overcoming many challenges. I also want to acknowledge Gemma Daron for her insightful advice, which contributed to shaping the direction of this project. Finally, I am deeply grateful to my family and friends for their unwavering support, which provided me with the strength and motivation to persevere.

Contents

1	Introduction	4
2	Optimisation and the Wind Farm Problem	5
2.1	Considerations for the Problem	6
2.2	The Wake Model	9
2.3	Wind Turbine Energy Production	12
2.4	Algorithms	14
3	Hill Climbing Algorithm	18
3.1	Method	18
3.2	Results	21
4	Genetic Algorithm	29
4.1	Method	29
4.2	Results	33
5	Scaling the Model	41
5.1	Method	41
5.2	Results	43
6	Conclusion	49

1 Introduction

Wind power, as a rapidly evolving renewable energy source, offers a promising solution to the growing demand for clean, sustainable electricity while reducing dependence on fossil fuels. Due to its long coastline and good wind speeds, the UK is ideally situated to harness wind power [1], both on and offshore. Since the first commercial farm was established in 1991 [2], the UK government has been striving to improve the capacity and efficiency of wind energy, with 9 offshore farm contracts approved in 2024 [3]. As of 2023, the energy produced from offshore wind makes up 17 percent of total generation in the UK, accounting for 49TWh of the total 316.8TWh demand in that year [4, 5].

To ensure these farms are generating the maximum amount of power, the layout of these turbines is put through a process called optimisation, which evaluates the best positions based on the factors impacting power production. These factors, the algorithms of optimisation and proposed optimal layouts are presented in this report, along with a novel approach to scaling optimisation algorithms to determine layouts that would generate the additional 50TWh required to power all of the homes in the UK, currently produced through means other than offshore wind [5]. Since this approach is based on theoretical calculations however, we advise that the results are only a hypothetical representation of optimal layouts and may not be applicable to real-life wind farms.

2 Optimisation and the Wind Farm Problem

Mathematical optimisation is an area of applied mathematics that explores methods used to optimise a function under some constraints. Optimisation problems consist of three components: the objective function(s) to be optimised, the decision variables, and the constraints which control the range of each variable [6]. Designing farm layouts that maximise power production whilst considering factors affecting the turbines is known as the Wind Farm Layout Optimisation Problem (WFLOP). Most solutions to this problem consist of optimally positioning the turbines within the wind farm so that the wake effects are minimised [7]. However, purely considering the wake effect can lead to limitations in the use of the layout solutions as well as potentially removing possible solutions from the search space. It is therefore important to consider the scenario of the problem to apply appropriate considerations to it (discussed further in Section 2.1).

The objective function represents a value to be minimised or maximised. In the case of the WFLOP this is usually to maximise power, or minimise the cost per energy (£/MWh) produced and when calculated separately is known as single-objective optimisation. However, since the WFLOP is multifaceted, only using a single objective would be insufficient in representing the problem, and so it is useful to extend the approach to multiple objective functions in order to evaluate layouts. This is known as multi-objective optimisation, which finds a set of results based on the optimal trade-offs between the objectives (as opposed to single-objective optimisation, which searches for a best singular solution). This set of solutions is known as Pareto optimal solution set, or Pareto front. In this report, many of the solutions discussed refer to *Optimization of wind turbine positioning in large windfarms by means of a genetic algorithm* [8] in which the cost to power ratio under three different wind scenarios was minimised using a genetic algorithm. Although the solutions presented by the authors, Mosetti et al., have been improved and expanded on, it is a useful control for both designing the optimisation algorithm and comparing the results with those from

other authors.

2.1 Considerations for the Problem

As with many real-world optimisation problems, various factors influence the efficiency of wind farm layouts, which can significantly affect solutions. The most prominent of these factors is the wind itself, particularly its direction and intensity [8]. For optimal energy production, turbines must be oriented to face into the prevailing wind, necessitating a consideration of both single and multiple wind directions in the layout design. This issue is further complicated by the wake effects produced by turbines, which can alter both the direction and velocity of the wind downstream, thus impacting the performance of nearby turbines.

The type of wake model must be selected to match the scenario of the turbine placement, in order to obtain accurate results from the optimisation. Additionally, wind speed and its availability are subject to changes in season, geography and meteorology [9], which introduces the possibility of location-dependent fluctuations in power output. These considerations could be implemented through historical wind data in prospective areas, although this would not account for unpredictable events such as storms. Models relying on previous data may not be able to fully encapsulate these uncertainties, suggesting that a model that incorporates real-time forecasting alongside the data could improve the robustness of the results.

Although wind is the main consideration, it is important to note that the set up of the farm itself is also a factor in optimisation processes. Mosetti et al. presented a method of using a discrete square grid of 100 possible turbine positions to arrange farm layouts, with each position being a set distance to employ the single wake model (discussed further in Section 2.2) [8]. They elected to use only one type of turbine in their calculations, along with neglecting additional costs in order to reduce the problem complexity. Although this is useful for theoretical layouts, it is not truly representative of real-life wind farms - most utilise turbines of varying characteristics and the farms

themselves do not have a consistent side length due to the space allocated – implying the results from Mosetti et al. are idealised compared to actual farms. Optimising the farm based on economic factors is useful for planned wind farms, but for hypothetical scenarios it would be difficult to produce ‘realistic’ solutions. Instead, wind farm efficiency would make a better measure of a good solution for a hypothetical area.

Mosetti et al. presents a simplified model for assessing the cost of the wind farm, which only considers the number of turbines [8]. They assume that if a large number of turbines is installed at one site, the cost per turbine can be reduced by $\frac{1}{3}$, with the cost per year equation being

$$\text{cost}_{tot} = N_t \left(\frac{2}{3} + \frac{1}{3} e^{-0.00174 N_t^2} \right), \quad (1)$$

where N_t is the number of turbines [8]. Since the focus of the paper was on the applications of genetic algorithms to the WFLOP, the considerations and scenario set for the problem are not true representations of a realistic model. A better representation of the cost of wind farms is known as the Levelised Cost of Energy (LCOE), which evaluates the total power produced by an energy source compared to its costs over the lifetime of the source [10]. The equation for LCOE is given by

$$\text{LCOE} = \frac{\sum_{t=1}^n \frac{I_t + M_t + F_t}{(1+r)^t}}{\sum_{t=1}^n \frac{E_t}{(1+r)^t}}, \quad (2)$$

where I_t , M_t and F_t are the annual costs (investments, maintenance and fuel respectively), E_t is the annual energy produced, r is the discount rate, also known as weighted average cost of capital (WACC) and n is the lifetime of the source. LCOE can be calculated for any energy source, making it a highly valuable metric for comparing different energy options. For example, it can help determine whether wind or solar power would be more effective at a specific site based on factors such as energy production and associated costs. However, it is important to note that LCOE does not account for fluctuations in the market value of energy, nor does it consider

the presence of pre-existing sources of energy generation (such as improving a pre-existing farm) [10]. Recognising this limitation, the layouts produced in this report do not reflect actual operating wind farms, and the energy market dynamics are not incorporated into the optimisation process. As a result, we can safely disregard the impact of market value changes or existing generation sources when interpreting the LCOE results in our analysis. This ensures that the focus remains on the technical optimisation aspects.

Such complexities highlight the need for a comprehensive approach in the design and optimisation of wind farm layouts to ensure maximal energy generation. In order to reduce computational time and produce layouts that can be compared to other papers in the subject, we will reduce the problem to the following considerations:

- The single-objective optimisation (Hill Climbing Algorithm) will maximise power output, to provide an initial point of reference for the results.
- The multi-objective optimisation (Multi-Objective Evolutionary Algorithm) will be used in two separate calculations. The first aims to maximise power output and minimise cost and the second aims to maximise power output and minimise farm area.
- The economic models used will be from Mosetti et al. [8] and the LCOE equation, with values taken from the *Guide to Offshore Wind Farms* by BVG Associates from 2019 [11].
- Only one wake model will be considered (see Section 3.2).
- Only one type of turbine will be considered, and the characteristics associated with the turbines are adopted from Mosetti et al. [8].
- The cell size of the optimisation grid will be 200x200m, adopted from Manikowski et al. [12].

By constraining the problem as above, we enable a well-structured approach to generating solutions to the WFLOP. There are many limitations to these constraints however, many of which are addressed later in the report. The economic models used rely on assumptions made in 1994 and 2019, which may not reflect the most current economic conditions or local variations in energy prices and construction costs. Moreover, the optimisation ignores real-world constraints such as environmental regulations and logistical challenges, which can greatly affect the feasibility and cost-effectiveness of the designs. These limitations suggest that, while the optimisation provides useful insights, the results should be interpreted with caution and may require further refinement with more comprehensive models and real-world data.

2.2 The Wake Model

To ensure that the results of the optimisation can be reasonably compared to actual wind farms, we must consider the effects of wake from the turbines. As wind passes through the rotor disc of a turbine, its speed decreases as kinetic energy is absorbed through the blades [13]. This can affect the performance of other turbines located behind the first, as they receive slower, more turbulent wind, which can lead to reduced energy capture and efficiency. This is known as the wake effect. There are multiple models, both numerical and engineering-based, that can be applied to calculations to evaluate these effects and are key to creating an optimal wind farm layout with limited space. In this report, we will be utilising the kinematic wake model first developed by N. O. Jensen in 1984, known as the Jensen wake model, then later improved to the Katic-Jensen wake model in 1986 [14]. This model is often used for the WFLOP due to having good accuracy despite its simplistic format [13]. The model considers both individual turbines and the crossover effects of multiple turbines in a farm (single wake model and multiple wake model, respectively).

For the single wake model the formulation is as follows. Start with the wake decay constant k ,

$$k = \frac{0.5}{\ln(\frac{z}{z_0})}, \quad (3)$$

where z is the height of the turbine and z_0 is the roughness of the terrain surrounding the turbine. Then the axial induction factor a is calculated by

$$a = \frac{1}{2} \left(1 - \sqrt{1 - C_t} \right), \quad (4)$$

where C_t is the thrust coefficient. This is then used to calculate r , the radius of the wake region behind the turbine; that is,

$$r = r_0 \sqrt{\frac{1 - a}{1 - 2a}}, \quad (5)$$

where r_0 is the radius of the rotor. From the above, we can calculate v , the wind velocity behind the turbine:

$$v = v_0 \left[1 - \frac{2a}{(1 + \frac{kx}{r})^2} \right] \quad (6)$$

A diagram of the Jensen wake model can be seen in Figure 1, with labelling to illustrate how the equation is derived. Katic et al. proposed the model for multiple wakes as the squared sum of energy deficits at each wake [15], with the resulting velocity calculated from

$$\left(1 - \frac{v}{u} \right)^2 = \sum_{i=1}^N \left(1 - \frac{v_i}{u} \right)^2, \quad (7)$$

where u is the wind velocity and N is the number of turbines in the farm.

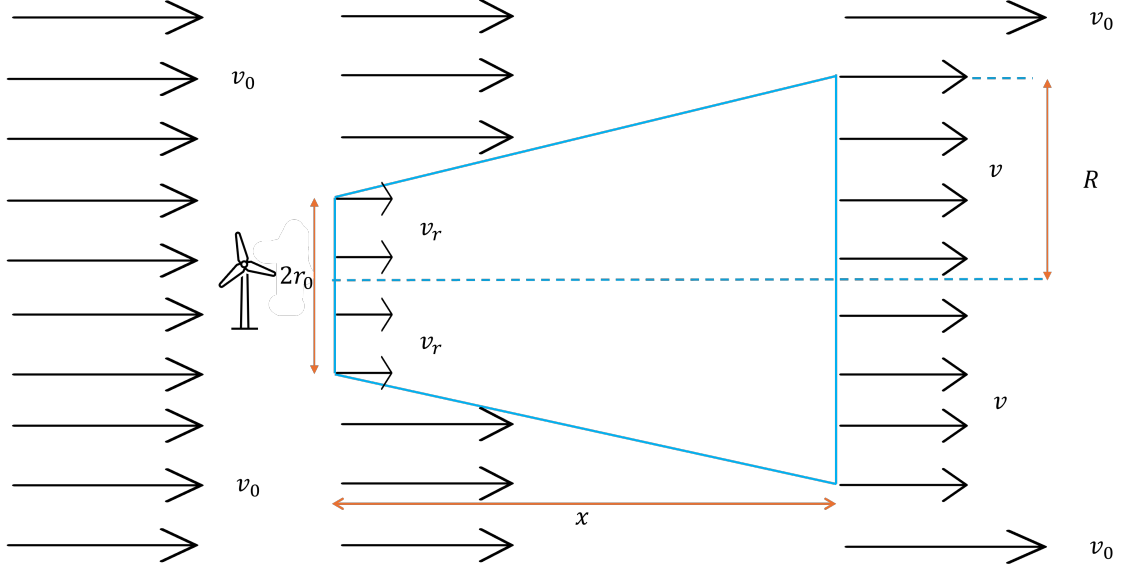


Figure 1: Visual representation of the Jensen wake model.

Manikowski et al. presented a combination of this squared sum equation and the wake velocity equation [12], which considers the partial wake effect on neighbouring turbines as

$$v = v_0 \left[1 - \frac{2a}{\left(1 + \frac{kx}{r}\right)^2} \left(\frac{A_0}{A_i} \right) \right], \quad (8)$$

where A_0 is the overlapped area between the rotor and wake areas, and A_i is the region swept out by the rotor in the i th turbine. Due to the arrangement of the grid in our results, we will only be considering the formula presented by Jensen et al. as the turbines will have a minimum separation distance of 200m (at least 5 rotor diameters apart), and therefore the single-wake effect applies.

The values of the variables that will be used in the above equations are taken from the paper by Mosetti et al., in order to make our results comparable to other papers which have also adopted these characteristics. These can be seen in Table 1.

Property	Value
k	0.0943695829
z	60 m
z_0	0.03
a	0.3267949192
C_t	0.88
r_0	20 m
ρ	1.225 kg/m ³
C_p	0.39
A	1256.64 m ²

Table 1: Wind Turbine properties proposed by Mosetti et al.

Although the Jensen wake model is widely used, it does have its drawbacks. Due to the wake deficit being based on a top-hat distribution and its simplicity in describing the velocity profile and wake decay, using this particular wake model can lead to results that do not wholly represent reality [16]. This does not discredit the model entirely, however; in fact, in a report conducted by Jeon et al. where four wake models were compared, it was concluded that the Jensen wake model worked best under set wind speeds [13]. As we will be using a singular wind speed for the optimisation, the Jensen wake model is well suited to our calculations despite its limitations.

2.3 Wind Turbine Energy Production

The energy produced by wind turbines is dependent mainly on the turbine capacity, wind speed and direction, which is why incorporating the wake effect into calculations is essential. Manikowski et al. presents a power output equation based on the power curve presented by Mosetti et al., which forms a basis for the optimisation in this report [8, 12]:

$$P = \sum_{i=1}^{N_t} \frac{1}{2} \rho A C_p v_i^3 \approx \sum_{i=1}^{N_t} 0.3 v_i^3 \quad (9)$$

Here, ρ is the air density, C_p is the power coefficient and A is the area swept out by the rotor blade.

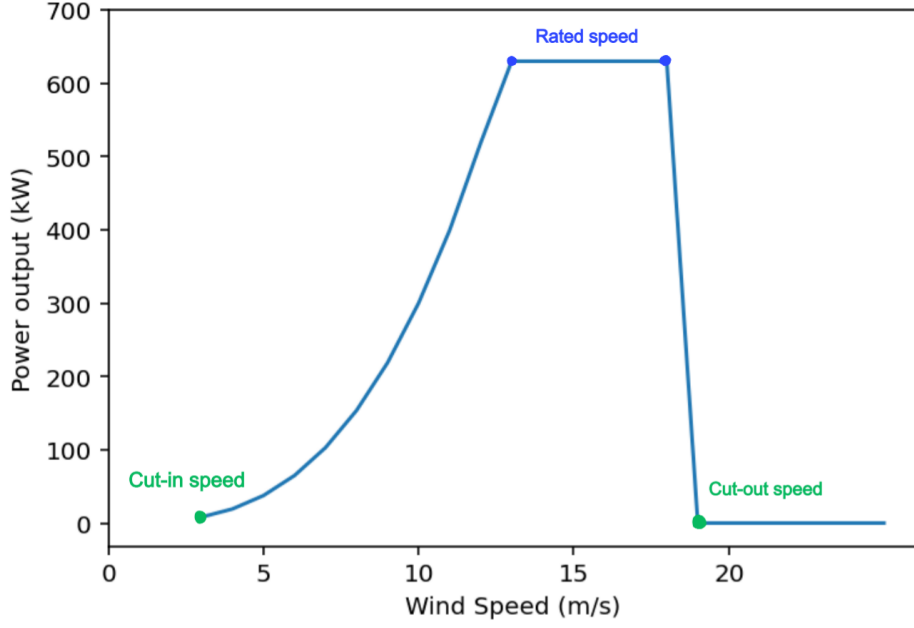


Figure 2: Power curve based on the values in Table 1.

For wind speeds below 2.3m/s and above 18m/s (the cut-in and cut-out speeds respectively), the turbine will not produce any power, and between 12.8m/s and 18m/s (the rated speed) the turbine will produce 630kW of power. Otherwise the power output is calculated from the above equation (see Figure 2). The distribution of the wind speed also impacts energy production. For the initial optimisation, we will only consider a single speed and direction following the first case presented by Mosetti et al [8]. Then, for the latter calculations, we will be using the distribution of directions for a single speed, also presented by Mosetti et al, in order to utilise a more sophisticated wake calculation and generate more accurate results. This distribution can be seen in Figure 3. Although this approach reduces the computational demands of the optimisation, it generalises the impact of wind speed distributions and directional variability, which are critical in real-world applications, highlighting the scope of the WFLOP and what future studies should consider.

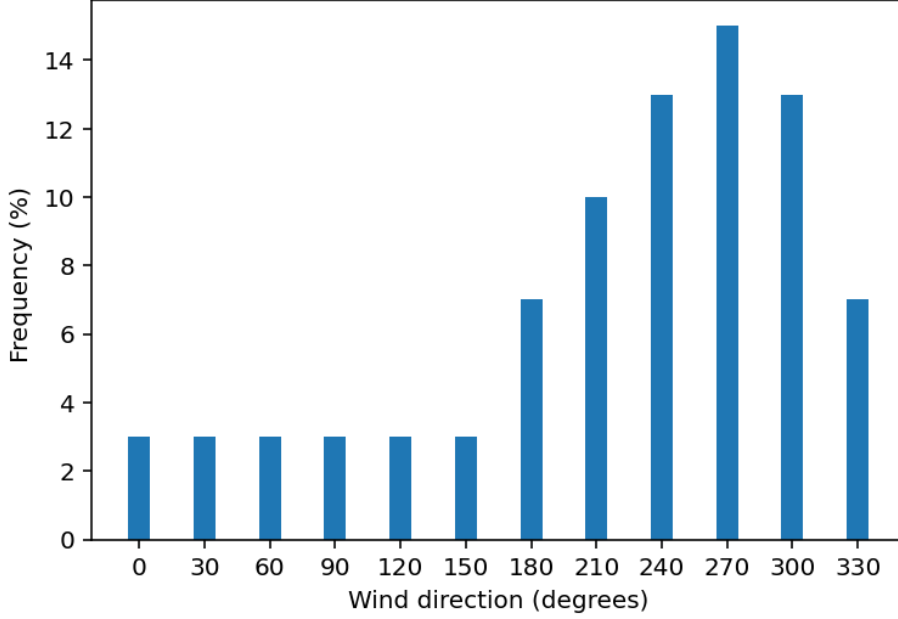


Figure 3: Wind direction distribution for 12m/s wind speed.

2.4 Algorithms

Algorithms are a set of instructions and rules that are followed in order to solve a problem. In the context of optimisation, these algorithms are iterative processes that converge to a stable solution, with the aim to obtain the optimal solution to the problem [17]. In order to select an appropriate algorithm to find solutions to the WFLOP, an understanding of the types of algorithms is needed.

Starting with a simple single-objective algorithm, the Hill Climbing Algorithm (HCA) is a local search algorithm that starts with an initial solution and makes iterative changes to it in order to improve the solutions. It is derived from the idea of the process one takes to climb up a hill and each step is intended to get one closer to the peak of the hill (assuming a maximisation problem), although this may lead to a different plateau. These are known as the global and local optima, respectively. For optimising wind farm layouts, the process starts from a randomly generated layout and compares with the values of neighbouring turbine positions. If the current layout is the best, then it is taken as the best layout; otherwise, the highest neighbour positions are used to replace the current layout. This iteration is repeated until it reaches the

peak [18] - in our case, this peak would be to maximise power output. Due to the potential of obtaining a local optimum, and lack of diversity in the search space, the HCA alone would not be a suitable method for this optimisation problem, but will be useful for evaluating the results of the genetic algorithms, similar to the approach by Manikowski et al. [12].

It is interesting to note that the results obtained by Manikowski et al. show that the HCA performs better than a more sophisticated algorithm in the single-objective optimisation case. However they conclude multi-objective optimisation is a better representation of the WFLOP, as it yields more balanced results by considering multiple objectives [12]. In this report, we will be using the HCA to optimise the 10×10 grid layout of turbine positions but not for the size scaling (as it is a multi-objective problem).

Genetic algorithms were first developed by John Holland, and are inspired by the processes of natural selection [19], where the fittest individuals in a population are selected to produce the next generation. Stochastic algorithms make a good fit for the WFLOP, as they use randomness and more sampling of the objective function, which can reduce the possibility of finding local optima [20]. Population algorithms are stochastic and maintain a population of solutions that are used to explore and locate optima. Examples of these algorithms that are used in the study of the WFLOP include particle swarm optimisation, genetic search and differential evolution [21, 22, 23].

Particle swarm optimisation was first devised by Erberhart & Kennedy in 1995 and is based on the observed swarm behaviour in nature, such as with birds or schooling fish [24]. One bird, for example, has a limited range of observable area, but with multiple birds they can be aware of a larger area. In the algorithm, a population of solutions (referred to as particles) move around a search space according to a formula on their velocity and position. The movement of these particles is influenced by its best local position, the best local position of the other particles and the best

known positions in the search space, which causes the particles to ‘swarm’ around the best solutions. Due to its fast convergence to single-objective solutions, and low computational requirement, it makes an effective method when applied to the multi-objective WFLOP, with Yin et al. stating it had “... high potential in practical applications” when applying it alongside other genetic algorithms [25].

In genetic algorithms, the process of natural selection is simulated to produce generations of individuals representing possible solutions. These individuals have a fitness score, which shows their ability to ‘compete’ and an optimal value of this is sought. Individuals with a higher fitness value are selected as parents to the next generation, and those with the least fitness are removed to allow the population size to remain static. Each generation would therefore have an improved gene pool (better partial solutions) than the previous. Given a sufficient runtime allowance, or number of fitness (or function) evaluations, a plateau can be reached where the offspring have no significant difference from the parent generation – the algorithm has then converged to a set of solutions to the problem. This method was notably used by Mosetti et al., although the models used in the paper are a more simplified version than what would be used for practical results, and they acknowledge that a more sophisticated windfarm and realistic economic model could be used [8].

The other key algorithm in this report alongside the HCA is the Non-dominated Sorting Genetic Algorithm II (NSGA-II). This is the improved version of NSGA, first proposed by Srinivas & Deb in 1994 [26]. As a multi-objective evolutionary algorithm, NSGA-II follows a similar process as described above. It has an initially randomly generated population, that is sorted by non-dominance and crowding distance into Pareto fronts, with each offspring and parent having assigned a fitness value and then are sorted again. Solutions are defined as non-dominated if there is no other solution that is better in at least one objective and not worse in the others in the population.

This method is utilised by Manikowski et al. alongside other Pareto-based algorithms (SPEA2 - Strength Pareto Evolutionary Algorithm 2 - and PESAI - Pareto

Envelope-based Selection Algorithm II), and they concluded that overall NSGA-II is the most appropriate algorithm for applying multi-objective optimisation to the WFLOP [12]. By contrast, in an analysis of NSGA-II conducted by Zheng & Doerr, they found that the algorithm is less effective when applying more than two objective functions [27]. This can be problematic, as the WFLOP can be characterised by many objectives. However, in this report we calculated additional objectives separately, which avoids this issue. In this report, NSGA-II will be used to optimise both the 10×10 grid layout of turbine positions and the size scaling optimisation to visualise the size of wind farm required to meet UK electricity demands.

3 Hill Climbing Algorithm

In this section, we present the application of the HCA to the WFLOP. The decision to utilise hill climbing was motivated by its simplicity and effectiveness as seen in the results presented by Manikowski et al [12]. However, as discussed in Section 2.4, there are limitations to the algorithm, such as the potential for convergence to local optima. Due to the algorithm having an aspect of randomness in its calculation, the results presented in this report may not fully match those of other authors. Although this does not discredit our results, it demonstrates how differences in implementation of the algorithms can impact final optimised layouts, and how evaluating different methods is key to improving results.

By adopting the characteristics from Mosetti et al. and Manikowski et al., we were able to create two different approaches to the HCA, with the aim to maximise the power output from the farm [8, 12]. In using two methods, we expect that there would be an improvement in the layouts from one to the other.

3.1 Method

Both versions of the algorithm were created using the Python programming language, but the set up of the calculations differs between them. In the first version, the single wake model is set up through the equations shown in Section 2.2 and the power generation calculation uses the equation from Section 2.3. The second version utilises an open-source wind farm modelling tool called *PyWake*, which is designed to simulate and analyse the wake effects in wind farms [28]. Going forward, we will be referring to these versions as Equation-based Algorithm (EBA) and PyWake-based Algorithm (PBA) respectively.

PyWake is primarily used for evaluating wind farm performance, such as calculating the annual energy production or analysing the impact of wind speed on a particular wind farm layout. We have utilised PyWake in our algorithm because of its relative

computational speed and adjustable parameters, making it straightforward to apply to the HCA and NSGA-II. Although using PyWake should produce results that represent the wake effects better than applying just the equations themselves, it is not a perfect modelling system. The library is designed to be modular, so the way each user implements PyWake may be different [28]. These differences in applications can lead to varying results from the models, so may not be truly comparable. However, for our results we have ensured that the correct wake model and power output calculations have been used, so that we may compare the layouts between each algorithm.

Creating the EBA involved using some of the standard Python libraries: Random, NumPy, Math, Matplotlib, and Timeit. The code itself consists of multiple functions that when run together create a step in the hill climbing process. The code process can be seen in the flow chart in Figure 4, and the code itself can be accessed through GitHub: <https://github.com/g-richardson-02/MATH3713-Project>. Due to it being constructed of standard libraries, the code was not complex to build, more so time consuming to correctly align all of the separate functions to ensure the outputs were correct. However, the same cannot be said for the PBA.

As PyWake is used purely for turbine wake modelling, it can be assumed that its target user base is researchers or engineers working in the wind energy sector. This means that for users without in-depth knowledge of wake modelling or Python experience would struggle to follow the documentation of the library. Combined with the variety of options for configuration and the lack of alternative resources, PyWake presents a steep learning curve for beginners. Despite this challenge, we were able to utilise PyWake to its full potential and the results of which will be discussed in later sections.

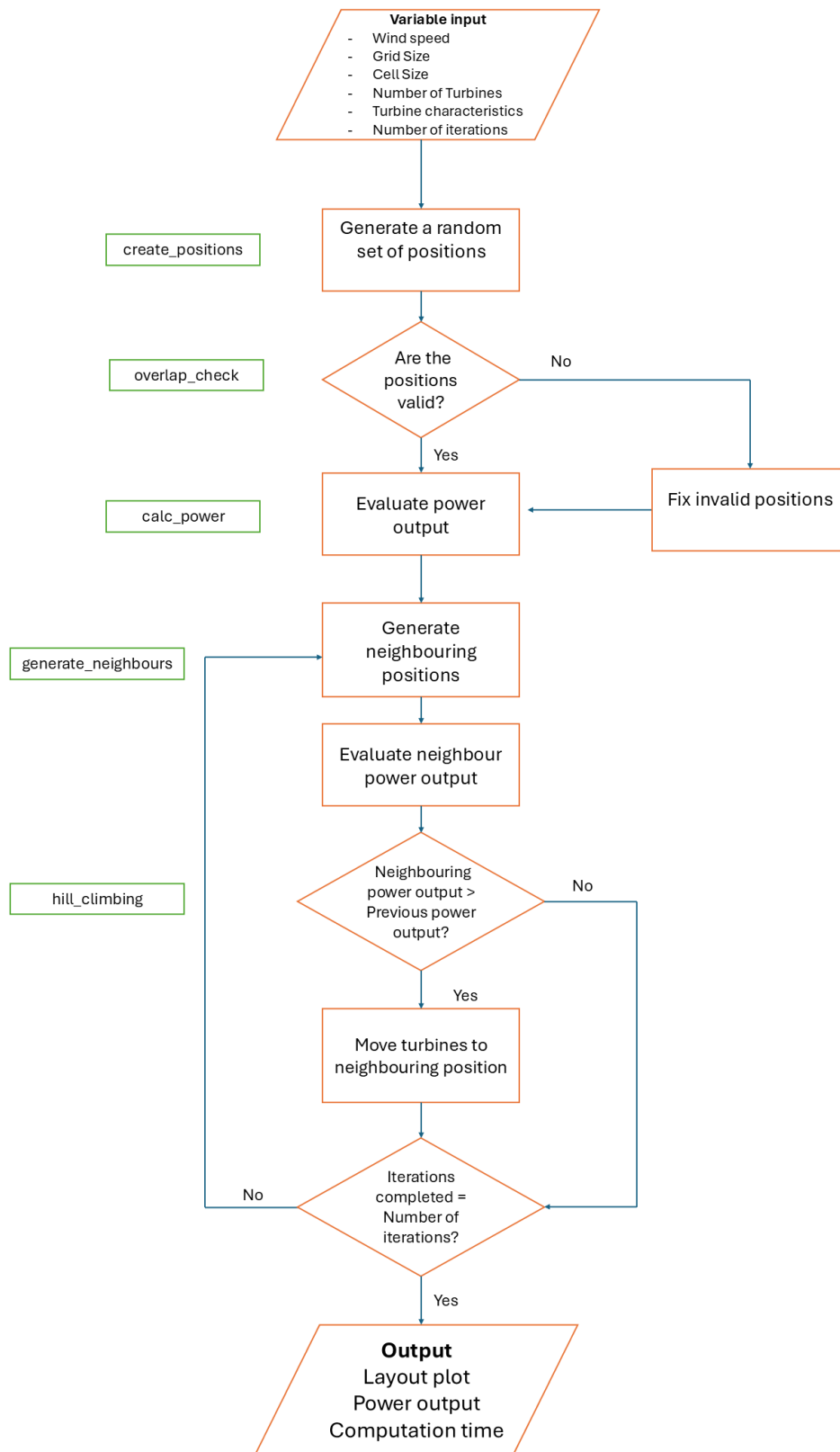


Figure 4: HCA code process with corresponding function names (green).

3.2 Results

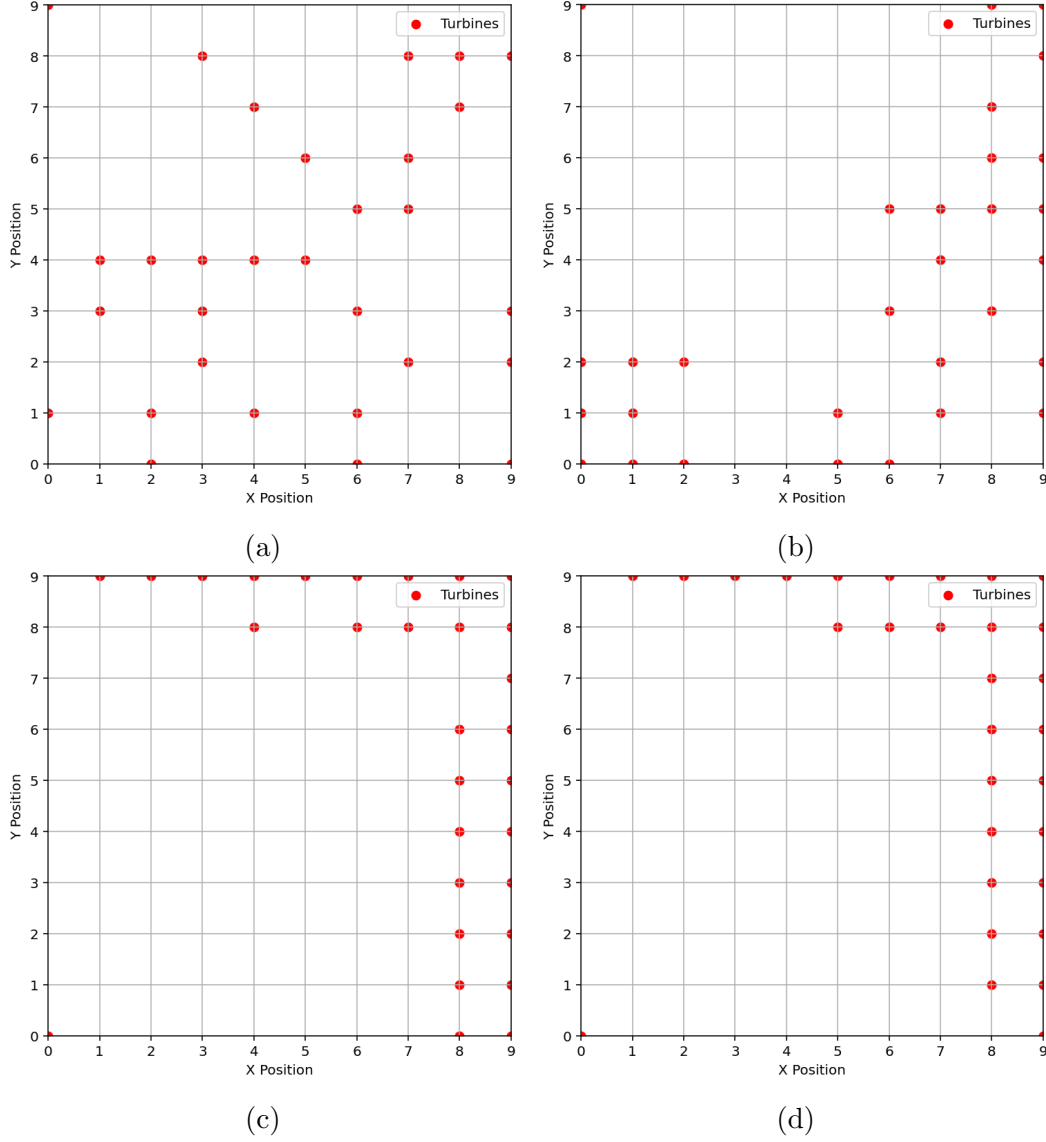


Figure 5: EBA layouts for 30 turbines: (a) 10 Iterations, (b) 50 Iterations, (c) 100 Iterations, (d) 500 Iterations.

The plots in Figure 5 are the layout results obtained when increasing the number of steps in hill climbing. The distribution of the wind (from Figure 3) is from the top of the plot to the bottom, with the wind speed set at 12m/s. To start, in Figure 5a we observe that the layout is mostly random, which is to be expected after only 10 iterations as the algorithm is limited on the number of neighbouring turbine positions it can search. After 50 iterations, we start to see the positions being pushed to the

outside of the grid although there is still some variety in the locations selected. Once we pass 100 iterations, the neighbour selection plateaus with the optimal layout having turbines placed around the edge of the grid. This is reflected in Figure 6, as past 100 iterations there is almost no improvement in the Annual Energy Production (AEP) value.

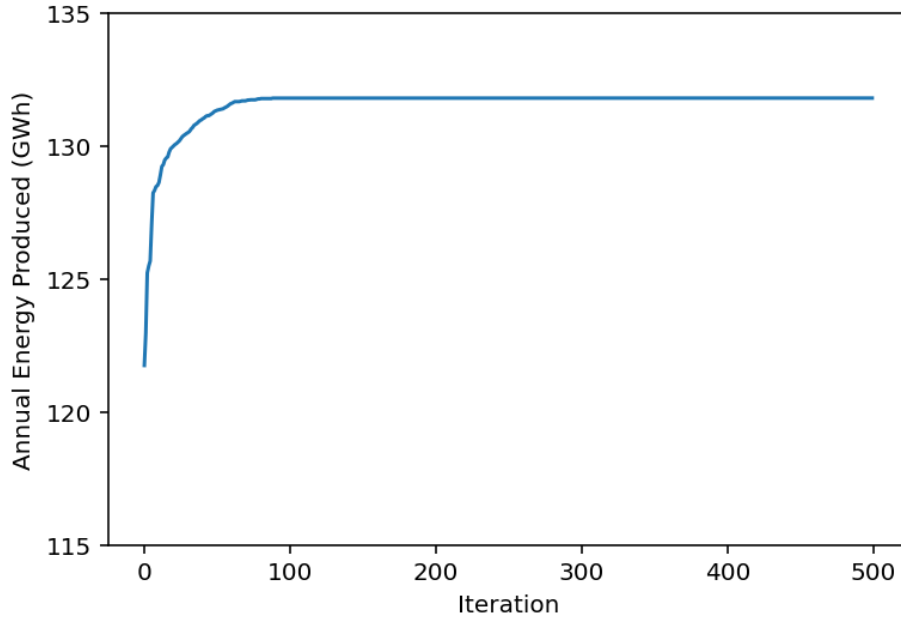


Figure 6: Comparison of power output against number of iterations for the EBA.

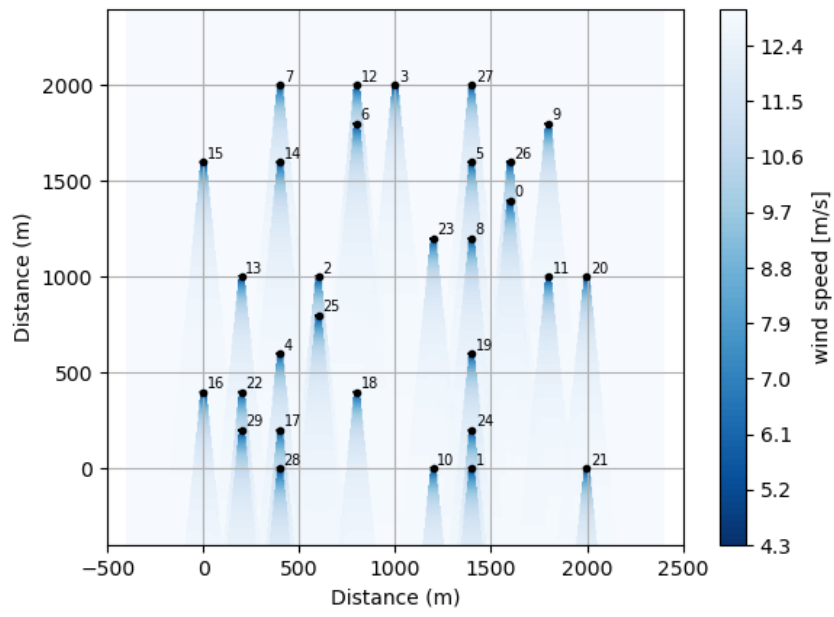
The calculated values for these layouts can be seen in Table 2. As expected, when increasing the number of iterations, the computational time of the algorithm increases. The optimal layout after 500 iterations has a power output of 131.81GWh annually, which is an improvement compared to the results presented by Mosetti et al. They achieved a power output of 108.41GWh using 26 turbines, with a turbine layout spread more along the top and bottom of the farm [8]. A key difference in these results is that the layout presented by Mosetti et al. contains only 26 turbines, as opposed to 30. As a result, the difference in the number of turbines likely contributed to the variation in power output. In the same way, these results were based upon data from more than 30 years ago, which would imply that if the authors had access to the same level of technology as we had when creating the algorithms, their results may match

or even exceed the performance of our layouts. In fact, Manikowski et al. presented improved results of the layouts from Mosetti et al., which showed an increase in the calculated power output [12]. The authors do not state what software they used in order to obtain these improved results, which may suggest that had they used a tool similar to PyWake, they would generate improved or more accurate values for the power output.

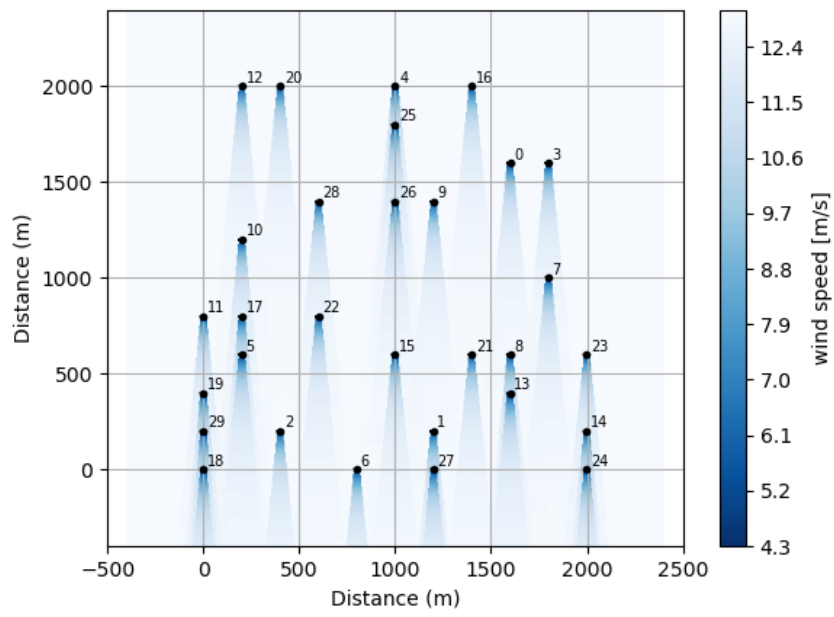
It is interesting to note that when applying the HCA to the multi-directional wind case, Manikowski et al. obtained a somewhat similar optimal layout, with turbines placed around the outside of the grid and some placed in the middle - obtaining a power output of 159.83GWh with 41 turbines [12]. Although not directly comparable to the layout we obtained due to the differing setup and number of turbines, it suggests that our turbine layout is effective and performs well under the given conditions. Further improvement of the EBA code, particularly the implementation of the equations, would likely generate more accurate results for the WFLOP.

Iteration	Power ouput in GWh	Runtime in seconds
10	128.39	1.08
50	130.85	4.79
100	131.65	8.99
500	131.81	45.61

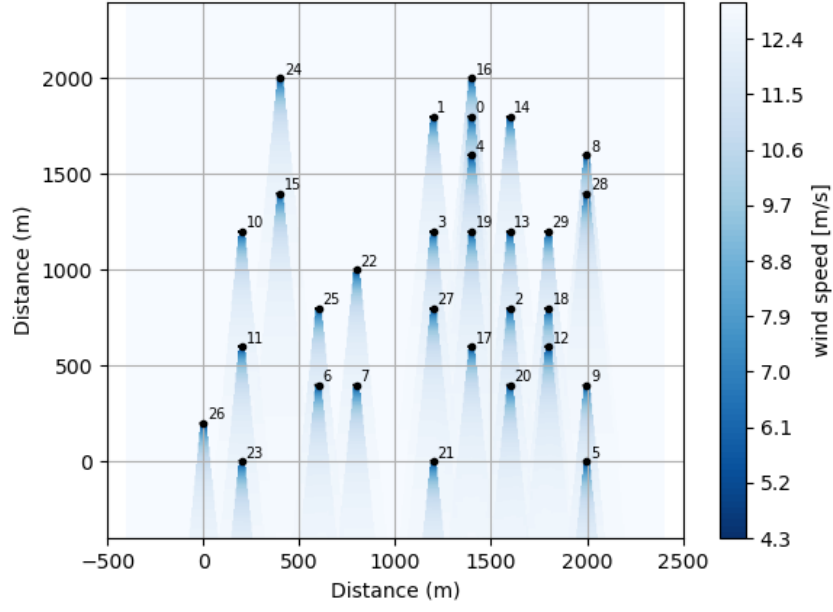
Table 2: Results of the EBA.



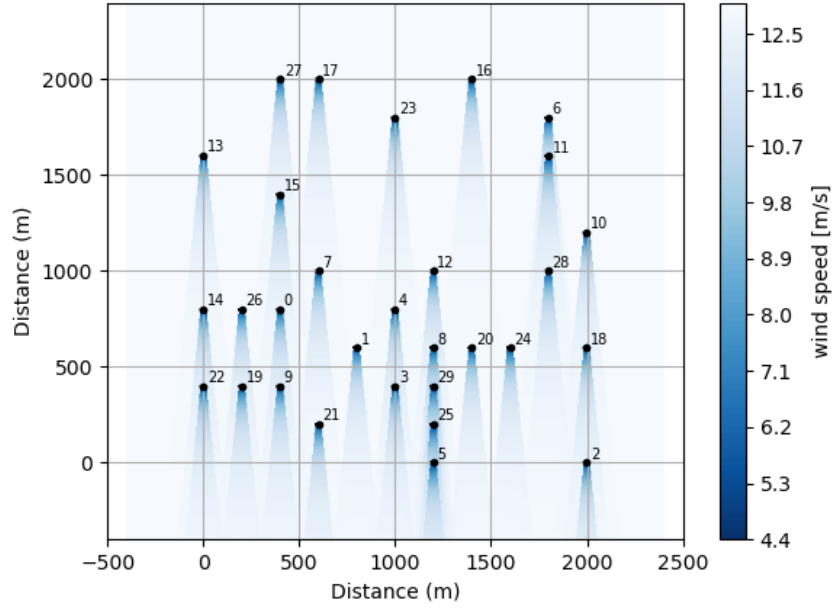
(a)



(b)



(c)



(d)

Figure 7: Hill-climbing algorithm layouts using PyWake: (a) 100 Iterations, (b) 500 Iterations, (c) 1000 Iterations, (d) 2000 Iterations.

The plots in Figure 7 are the flow maps of the turbine layouts produced through the HCA using PyWake. These results are on a continuous grid as opposed to the

discrete grid positions seen in the EBA, as PyWake enables us to position turbines at least 200 metres away from one another in any given direction. We decided to increase the number of iterations in the algorithm to 2000 because the AEP values were still rising up to that point. However, we aimed to avoid excessively long run times for minimal improvements beyond this threshold. This can be seen in Figure 8.

To start, with 100 iterations we observe that the layout is mostly groups of turbines placed around the grid with no exact symmetry, which would suggest that the solution is not fully optimal and still somewhat random. After 500 iterations we begin to see turbines being positioned more in lines behind and to the side of one another, with this trend continuing into the 1000 and 2000 iteration results. This could be attributed to a reduced wake effect on turbines placed next to each other. With a consistent wind direction, the wind speed impacting neighbouring turbines would not be affected, leading to a higher power output. This is similarly reflected in the layout presented by Grady et al., where the turbines are positioned in horizontal lines across the grid with several grid spaces between each line [22]. However, some variation in the positions remains, suggesting that further increasing the number of iterations might result in a more uniform layout.

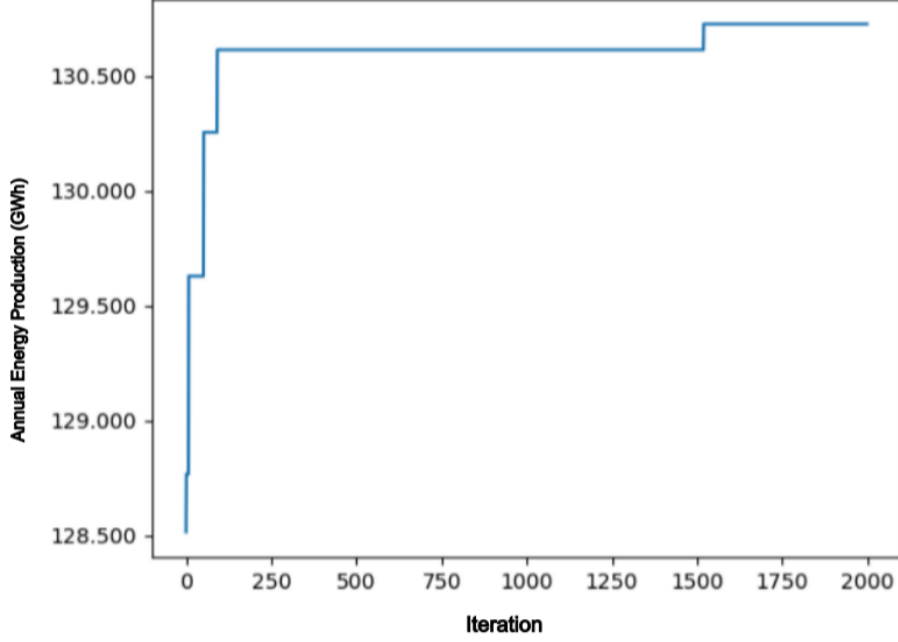


Figure 8: Comparison of power output against number of generations for the Hill-climbing algorithm utilising Pywake.

The calculated values for these layouts can be seen in Table 3. As with the EBA, we observe that increasing the number of iterations in the PBA also increases the power output and runtime. After 2000 iterations, we obtain a AEP of 130.73GWh for 30 turbines which is once again an improvement from the layout presented by Mosetti et al [8]. As well as this, it performs better than the horizontal line layout from Grady et al., as they achieved 125.36GWh for 30 turbines [22]. This would imply that, despite limiting the impact of the wake effect on the turbines by positioning them in uniform patterns, layouts with more variety in their positioning yield higher power production.

Grady et al. utilised a genetic algorithm for their results, which on the one hand suggests that a simpler procedure such as a HCA can achieve better results with less computational power required [22]. This lines up with the conclusion made by Manikowski et al., as their single-objective optimisation out-performed some of the multi-objective cases [12]. On the other hand, despite our result having a higher power output, it may not be as robust as the results presented by Grady et al. due to the inherent risk of a HCA getting stuck in suboptimal solutions. Despite this, it is

noteworthy that although there are many differences between genetic and non-genetic algorithms, both methods can produce similar layout structures.

Iteration	Power output in GWh	Runtime in seconds
100	129.54	42.19
500	130.34	219.93
1000	130.69	438.28
2000	130.73	883.46

Table 3: Results of the Hill-climbing algorithm using Pywake.

When comparing the two setups of the HCA, it is not entirely clear which performed best. The time to run the EBA is a lot less than the PBA, almost five times as fast to compute its results. This is to be expected, as extracting the different parameters and models used in PyWake is much more demanding than the libraries used to set up the EBA. The convenience of a shorter runtime does not outweigh the benefits of a more accurate model, however. Although the power output of the PBA was generally lower than the EBA despite having more iterations, the use of a more sophisticated tool in the PBA suggests that its results may be more representative of real-world conditions, and therefore more accurate. With features such as the ability to generate flow maps and a more realistic implementation of the Jensen wake model, the PBA offers a comprehensive solution for generating turbine layout designs.

4 Genetic Algorithm

In this section, we present the application of a genetic algorithm to the WFLOP, specifically NSGA-II. In the study of the WFLOP, this is a popular choice of algorithm due to its robustness, with authors such as Manikowski et al. using it as a comparison point against other algorithms [12]. Much like the HCA, NSGA-II does have some limitations, although as mentioned in Section 2.4 these often stem from using a large number of objective functions.

Pang et al. discuss in their report that poor performance when using NSGA-II is related to the existence of dominance-resistant solutions, wherein solutions have very good objective values for some objectives and very bad for others, which places them far away from the Pareto front [29]. Since our optimisation only has two objectives, our results should not be significantly impacted by dominance-resistance, and therefore we can disregard this issue. For our calculations, we continue to use the characteristics presented by Mosetti et al. to ensure meaningful comparisons between the algorithms. The aim of this section is to maximise the power output of the farm whilst minimising its associated costs, and we expect an improvement from the results of the HCA.

4.1 Method

As before, both versions of the algorithm were made in Python, with one using just the standard libraries, and the other using PyWake. We shall refer to these as genetic equation-based algorithm (GEBA) and genetic PyWake-based algorithm (GPBA), respectively. To set up the code to be able to apply NSGA-II we used a library called Distributed Evolutionary Algorithms in Python (DEAP) [30]. DEAP is designed to help users create and run evolutionary algorithms, particularly genetic algorithms, and similarly to PyWake it is very modular, allowing for easy customisation of the code. We elected to use DEAP over other similar libraries due to its straightforward documentation and implementation into Python - variables and functions were labelled

and explained in detail, and the process of setting up the genetic algorithms was uncomplicated. This, however, means that DEAP may have fewer high-level pre-built algorithm options when compared to other libraries. Despite this, it was able to comply with the requirements of this report.

The setup of the GEBA followed the same initial steps as seen in Section 3.1, with the addition of the DEAP library. Since this is a multi-objective optimisation, the algorithm aims to maximise power output first, then minimise the cost of the wind farm (using the cost equation presented by Mosetti et al. [8]). The process of the code can be seen in Figure 9. Despite also including the DEAP library, the code was not complex to set up (similar to the Hill-climb EBA) and only needed appropriate positioning of the DEAP toolbox functions. The setup of the PBA had fewer issues with the setup, due to transferring the majority of the code from the HCA. The main difference is the inclusion of the LCOE calculation as opposed to the cost equation presented by Mosetti et al. The values used for the LCOE were adopted from the Offshore Renewable Energy Catapult (OREC) from 2019 and can be seen in Table 4 [11].

It is worth noting that although the LCOE is a more effective method of evaluating the wind farm cost when compared to the method used in the EBA, there is a risk it underestimates costs. Moreover, in an article written for ENODA, an energy technology company, Williams & Domjan discuss that although LCOE is useful for cost-competitiveness across different energy sources, it often does not consider other external or system-wide costs [31]. Acknowledging this, we anticipate that our results may not convey the full extent of the cost to operate a wind farm and we will comment on this in the following sections.

Property	Value
Nominal turbine power	3 MW
Capital Expenditure (CAPEX)	£2,370,000 per MW
Operating Expenditure (OPEX)	£76,000 per MW
Lifetime	25 years
WACC	6 %

Table 4: Values used to calculate LCOE based on OREC 2019 [11].

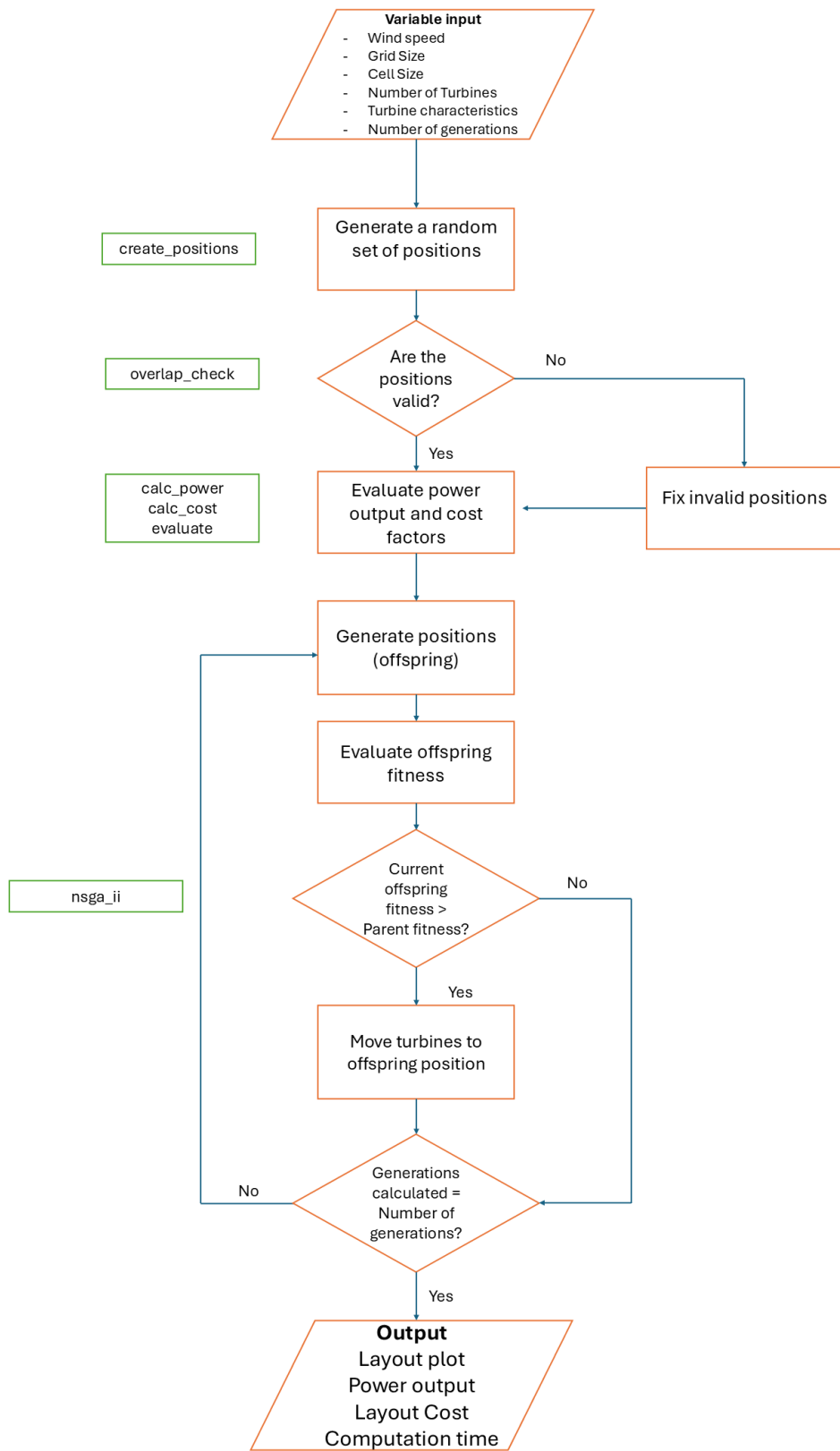


Figure 9: NSGA-II code process with corresponding function names (green).

4.2 Results

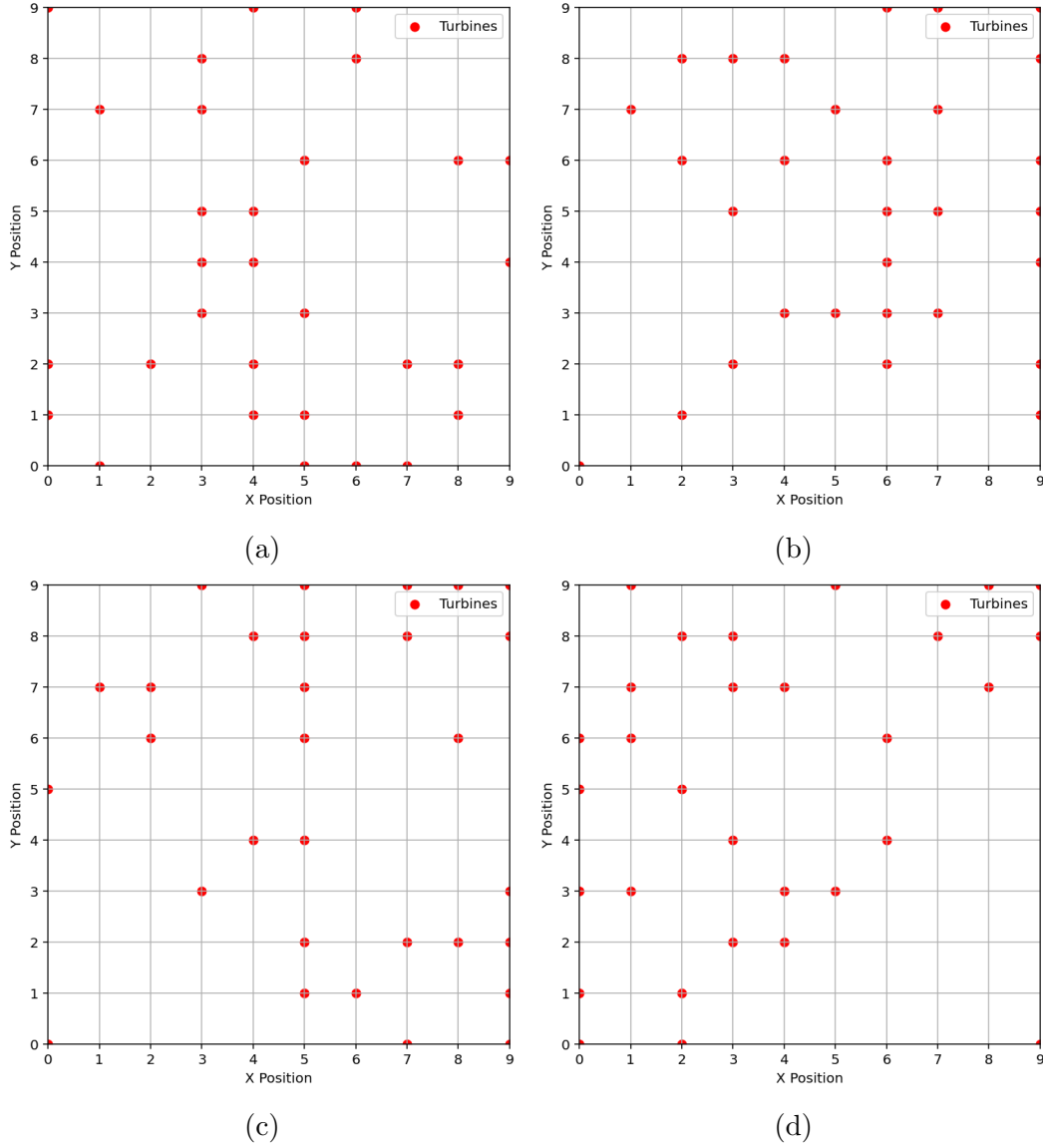


Figure 10: NSGA-II layouts for 30 turbines: (a) 10 Generations, (b) 50 Generations, (c) 100 Generations, (d) 500 Generations.

The plots in Figure 10 are the layout results obtained when increasing the number of generations of NSGA-II for the GEBA. In parallel with the HCA, after 10 generations, the layout remains random, as an insufficient number of generations have been computed. Between 50 and 100 generations, the turbine positions begin to align in more uniform patterns, although some variation still persists. This trend continues through to the layout after 500 generations. This suggests that a greater number of generations

may be required to further improve the solution, in contrast to the HCA, which had plateaued completely after 500 iterations. In Figure 11, there is an obvious plateau in solutions past 150 generations, which would imply that we have reached an optimal layout at 500 generations. It should be mentioned that the symmetry observed in Figure 11 is a consequence of utilising the cost equation presented by Mosetti et al. Since the number of turbines remains constant in these calculations, the cost-to-power ratio only varies when the power output increases. This approach is therefore inherently inefficient for assessing the cost impact, providing further justification for the use of LCOE instead.

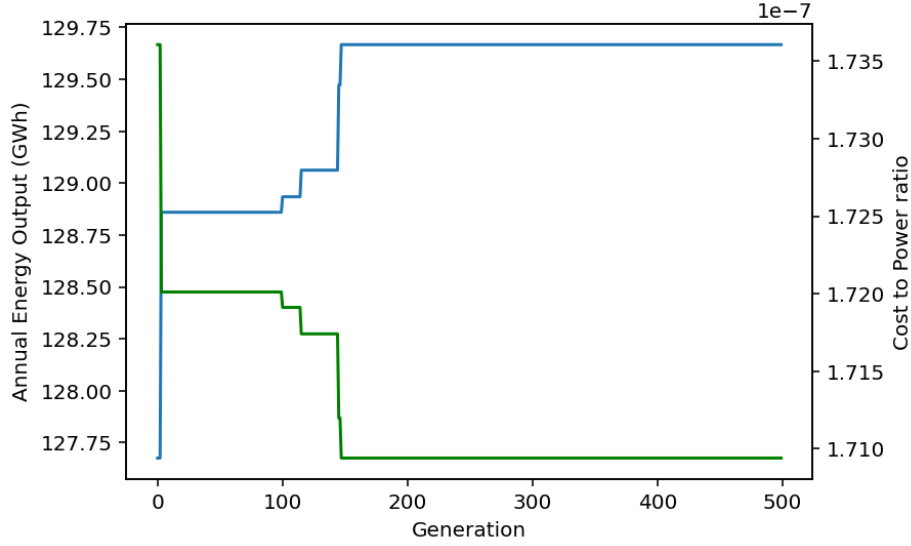


Figure 11: Comparison of power output (Blue) and cost-to-power ratio (Green) against number of generations for NSGA-II.

The calculated values for these layouts can be seen in Table 5. As expected, the runtime increases almost linearly as the number of generations increases. When compared to the hill climbing EBA, we see that using a genetic algorithm roughly doubles the runtime. This is reasonable, as the computational power required to run a multi-objective optimisation is greater than that of a single-objective case. When increasing the number of generations, the power output increases, and the cost-to-power ratio decreases; however, it is important to note that the power output after 500 generations is lower than that of the hill climbing EBA after the same number

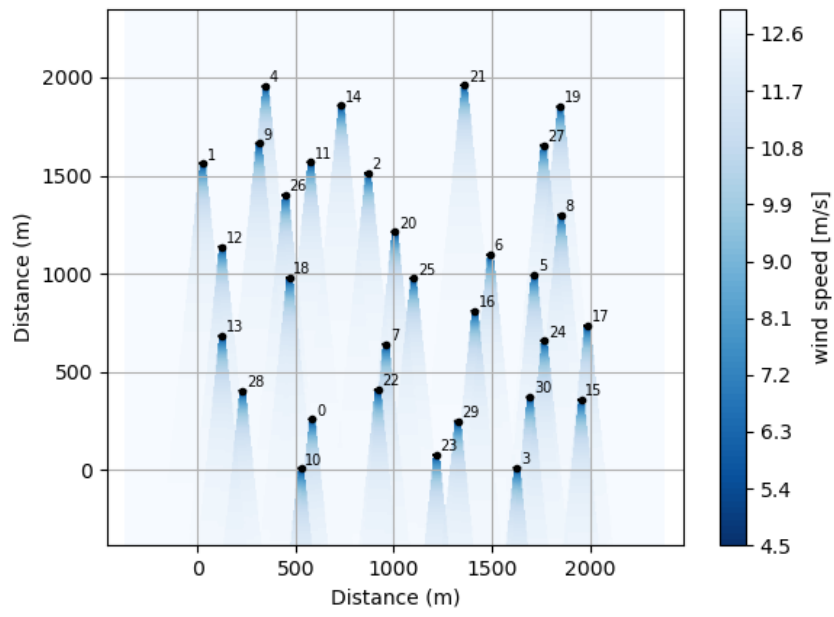
of generations. This is likely a result of the algorithm balancing the two objective functions, suggesting that the layout produced by this method may represent a more optimal solution than that of the HCA.

When comparing the results to those of Mosetti et al., a clear difference in the layouts emerges, particularly in the lack of uniformity in turbine positioning. Mosetti et al. presents that the optimal layout places turbines along the edges of the grid, a configuration similar to our HCA results and those from Manikowski et al. [8, 12]. Although our algorithm resulted in a broader distribution of turbine positions, only some turbines were located along the edges. This could indicate that further iterations might lead to results more like those from the HCA. However, since the layout presented by Mosetti et al. includes only 26 turbines, it is also possible that with a larger number of turbines, their layout would show a wider spread of turbine positions.

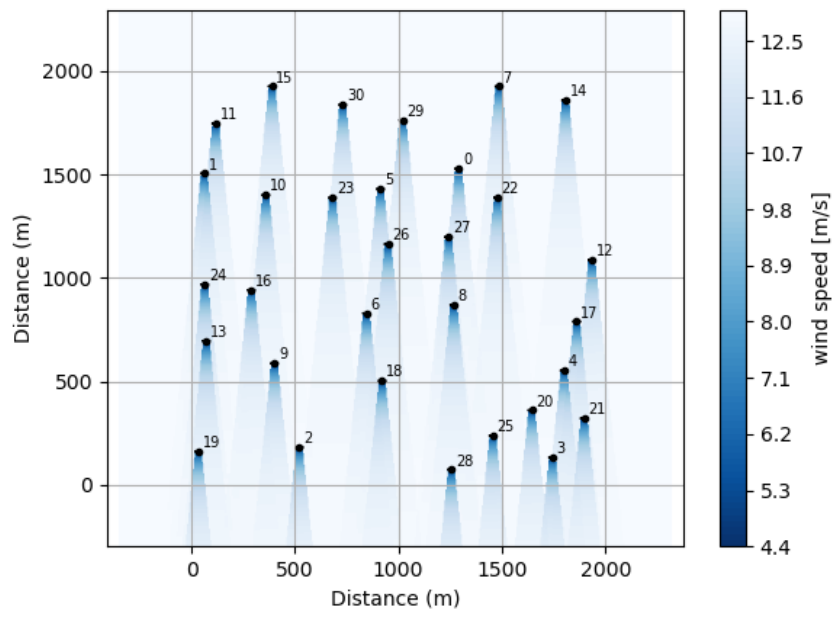
In terms of the objectives, we achieved a higher power output than Mosetti et al. after 500 generations. Converting their ratio into our result format gives a value of 0.18454 for their layout, which is higher than the ratio for our layout after 500 generations. This suggests that our layouts are more optimal, as we achieved both a greater power output and lower cost-to-power ratio. However, it is important to consider potential differences in the methods used by Mosetti et al. to generate their layouts, which may have contributed to the varying results.

Generation	Power output in GWh	Cost-to-Power ratio	Runtime in seconds
10	126.73	0.17493	2.27
50	128.36	0.17271	9.39
100	129.29	0.17147	26.19
500	129.67	0.17097	94.60

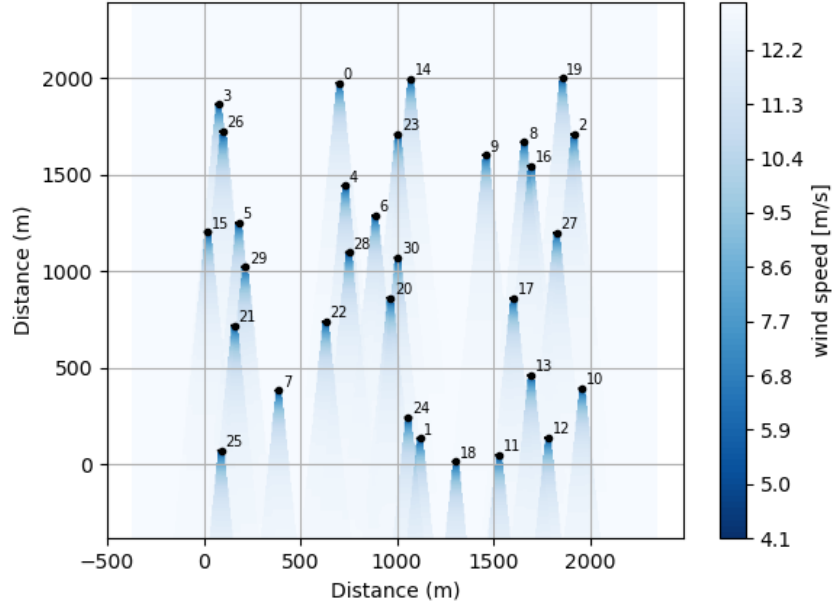
Table 5: Results of NSGA-II.



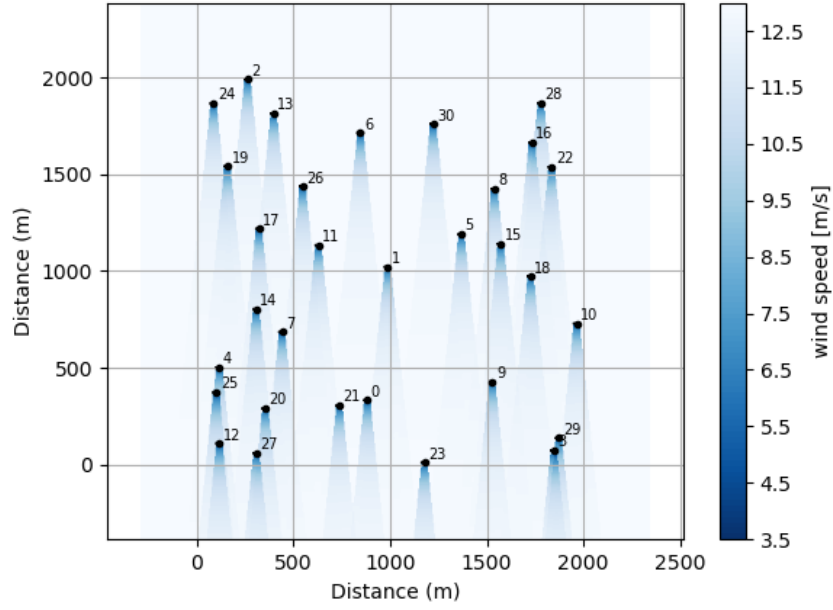
(a)



(b)



(c)



(d)

Figure 12: NSGA-II layouts using PyWake for 30 turbines: (a) 100 Generations, (b) 500 Generations, (c) 1000 Generations, (d) 2000 Generations.

The plots in Figure 12 are the flow maps of the turbine layouts produced using the NSGA-II algorithm with PyWake integrated into the calculations. As with the

HCA, we set a maximum of 2000 generations to minimise the algorithm computation time. However, as shown in Figure 13, further increasing this limit could lead to better results, since the previous best layout was after 1600 generations. After 100 generations, the positioning of the turbines is fairly spread out with no exact pattern discernable as expected from previous results. Between 500 and 1000 generations we see a transition to a more linear layout of horizontal lines and groups of turbines. Finally, after 2000 generations, the layout is once again spread out with fewer turbines located in the centre of the grid. This mirrors the comments made by Mosetti et al. as mentioned in the EBA results for this section, in which the optimal layout pushes turbines to the edges of the grid and implies that increasing the number of generations could position more turbines in this way [8].

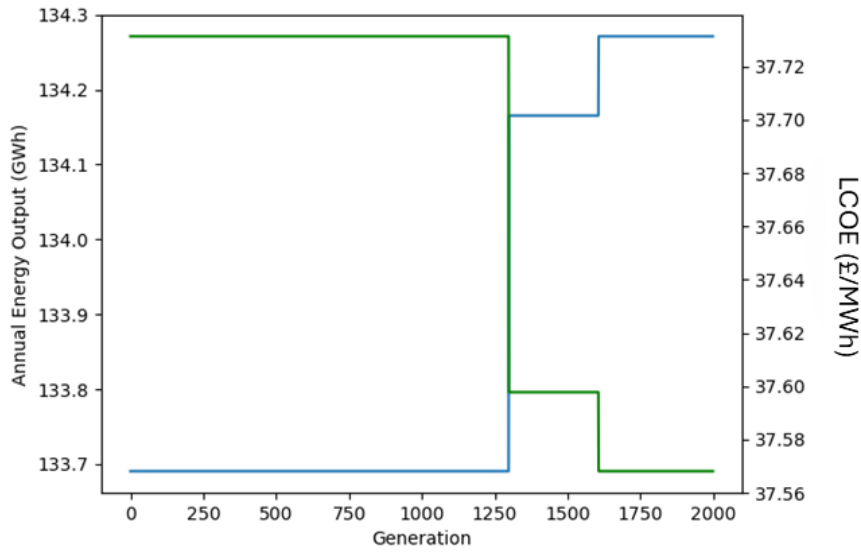


Figure 13: Comparison of power output (Blue) and cost-to-power ratio (Green) against number of generations for NSGA-II utilising Pywake.

The calculated values for these layouts can be seen in Table 6. The runtime of the algorithm is around five times longer than the PBA for the HCA, due to the higher demands of the DEAP library. 2000 generations took just under 50 minutes to compute, and when evaluating the objective functions, it is interesting to note that the layout produced after 1000 generations performed marginally better. This

could be due to the search space being sufficiently explored, leading to diminishing returns after 1000 generations. By adjusting the algorithm parameters (such as the mutation or the crossover rate), the performance could be improved to solve this issue. Additionally, the power output when using NSGA-II is higher than that of using the HCA for the PBA, which may be attributed to the HCA finding a local optimum. This gives further evidence that NSGA-II is a more appropriate algorithm for generating solutions to the WFLOP.

As LCOE is used for these calculations, it is difficult to compare our results to Mosetti et al. Since we have adopted the values for the LCOE from the OREC, it would instead be a suitable comparison. Their findings based on wind farms around Europe show a range of LCOE values between 50 and 80£/MWh for 2019 [11]. This is significantly higher than the results obtained through our calculations, which may be due to the idealised assumptions we have made about the factors affecting wind energy production.

In addition to this, from the UK Offshore Wind Report from 2023 we know that the average number of turbines in an offshore wind farm is 61, with an average turbine capacity of 5MW [5]. Since our layouts consist of 30 turbines, with a nominal capacity of 3MW, we would expect to have a lower LCOE in our results than actual farms. Fewer turbines would result in a smaller required investment in transmission lines and substations, as the smaller energy output puts less pressure on the grid infrastructure. Additionally, the initial capital cost would be reduced because there would be fewer turbines to install.

Another notable comparison for LCOE is the results presented by Gonzalez-Rodriguez et al., wherein they utilised NSGA-II to minimise LCOE and visual impact [32]. For 80 turbines, they had an LCOE between 58 and 63, which could be argued is a more accurate representation of a real wind farm as it falls between the values presented by OREC [11]. This further highlights the effectiveness of using NSGA-II to produce solutions for the WFLOP, and the potential to improve our own algorithm to achieve

more optimal solutions.

Generation	Power output in GWh	LCOE (£/MWh)	Runtime in seconds
100	133.71	37.73	223.93
500	133.84	37.68	877.92
1000	134.40	37.53	1465.04
2000	134.27	37.57	2921.54

Table 6: Results of NSGA-II utilising Pywake.

When comparing the two versions of the genetic algorithm, there is a clear improvement in power output, with the GPBA showing a 4.42GWh increase over the GEBA results after 100 generations. However, this comes at the cost of a significantly longer runtime. As noted earlier, comparing the Mosetti et al. cost equation to the LCOE is challenging, so future calculations may yield more meaningful results if only one of these methods is used. Despite this, based on the outcomes from both the HCA and NSGA-II, we conclude that using PyWake for optimising wind farm layouts provides more accurate and effective results than relying on standard equations alone. Therefore, we have incorporated it into the area scaling calculations, which shall be discussed in the following sections.

5 Scaling the Model

Advancements in wind turbine technology have made it difficult to precisely estimate the number of turbines or wind farms needed to generate the roughly 50 TWh required to supply energy to all homes in the UK [5]. Given that 2,766 turbines provide enough energy to power half of the homes in the UK, it can be estimated that approximately 5,600 turbines total would be required to meet the demand [5]. For perspective, Hornsea 2 (currently the largest offshore wind farm in the world) consists of only 165 turbines, and has a capacity of 1.32 GW (11.6 TWh annually), suggesting that the actual required number of turbines is less than estimated [33]. This is most likely due to the improvement of turbine capacity, leading to more efficient farms with less turbines.

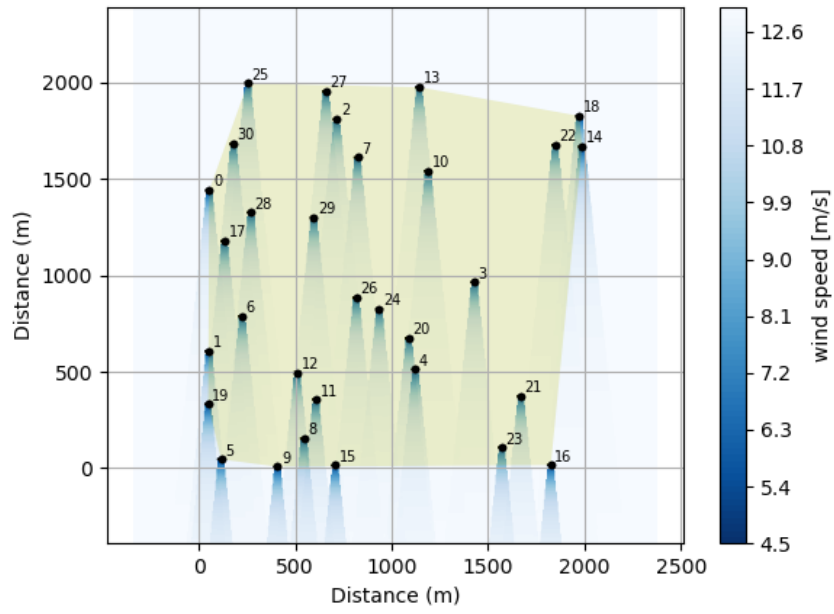
We aimed to explore whether we could use PyWake and NSGA-II to create optimal layouts that maximise power output while minimising farm area. The goal was to use this to estimate the potential wind farm area and the number of turbines needed to power all homes in the UK. Optimising farm layouts in this way is rarely considered, so it would be difficult to compare with other authors; therefore, we are using Hornsea 2 as a comparative case.

5.1 Method

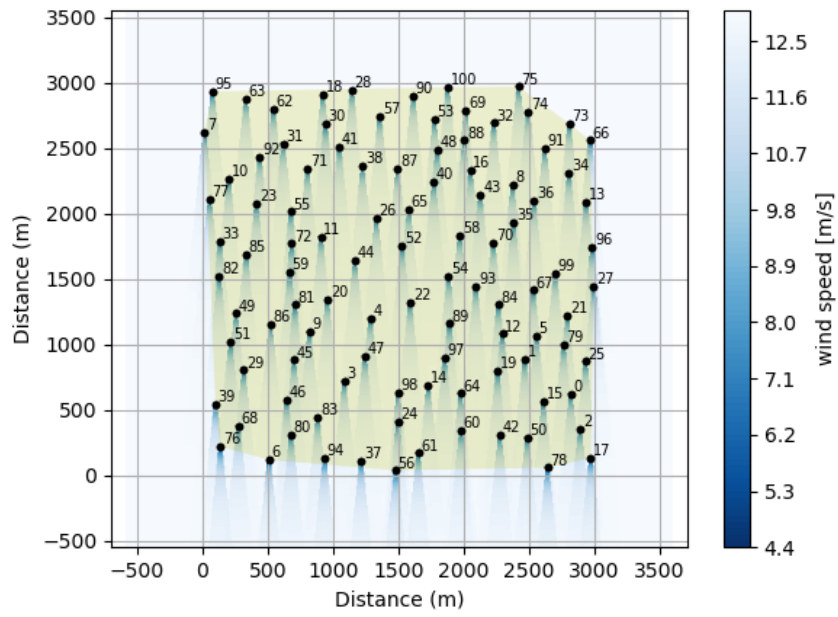
The process of the code followed the same initial setup as the GPBA, the main difference being the use of the SciPy library. This enabled the area size to be calculated on the layouts, taking into account the furthest positioned turbine on each side. This was then used in the algorithm instead of the LCOE, and the boundaries of the farm were made visible on the layout plots. Since this is a fairly novel method, we would not expect the results to be wholly representative of the WFLOP but rather an insight into how mathematical optimisation may be useful for the future of wind farms management.

NSGA-II was used to maximise power output and minimise the farm area. To achieve a higher power output, we increased both the number of turbines and the length of the grid boundaries. However, to reduce computation time, we limited the number of generations to 100, as this provided the best balance between results and runtime. We began with 30 turbines, then increased the count to 100, calculating several results with progressively larger grid sizes, followed by 500 and, ultimately, 1000 turbines. The wind distribution follows the same as previous sections (see Figure 3), as with the wind speed (12m/s). The results of these calculations are presented in the following sections.

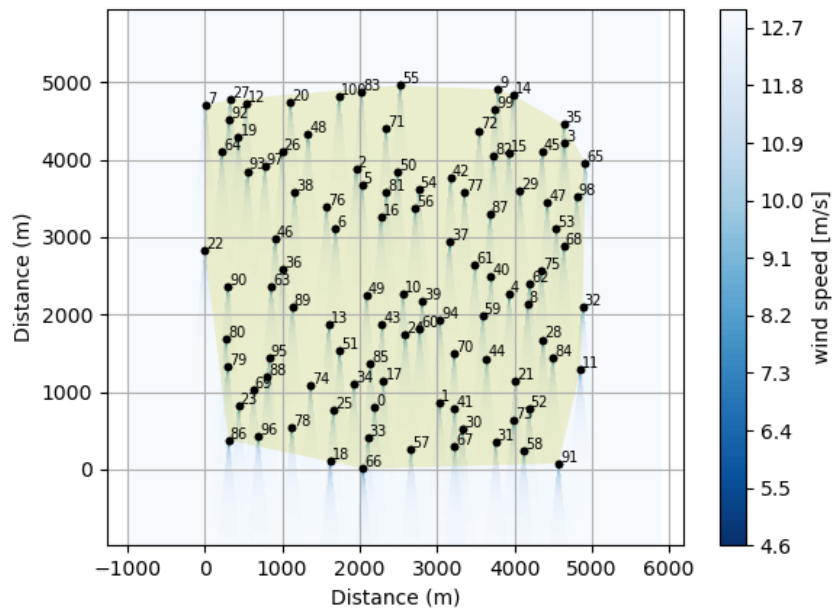
5.2 Results



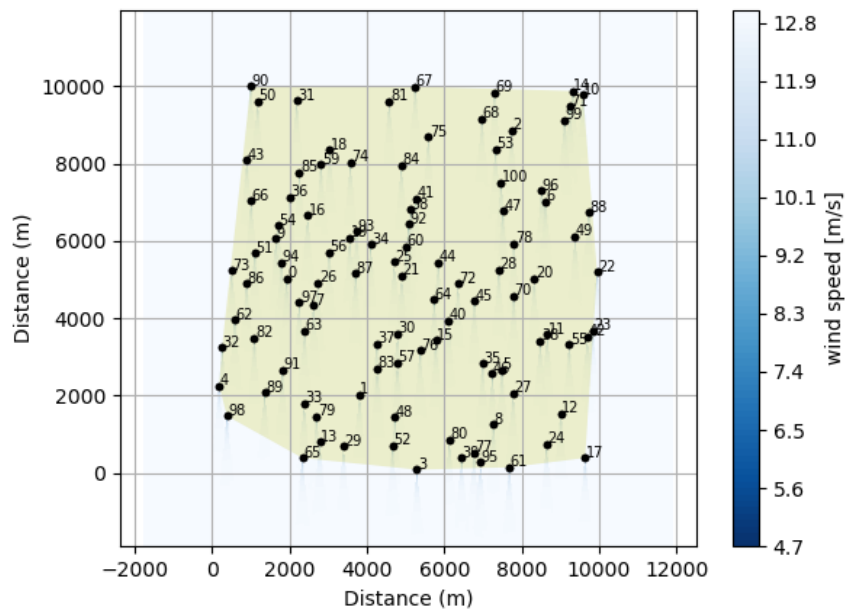
(a)



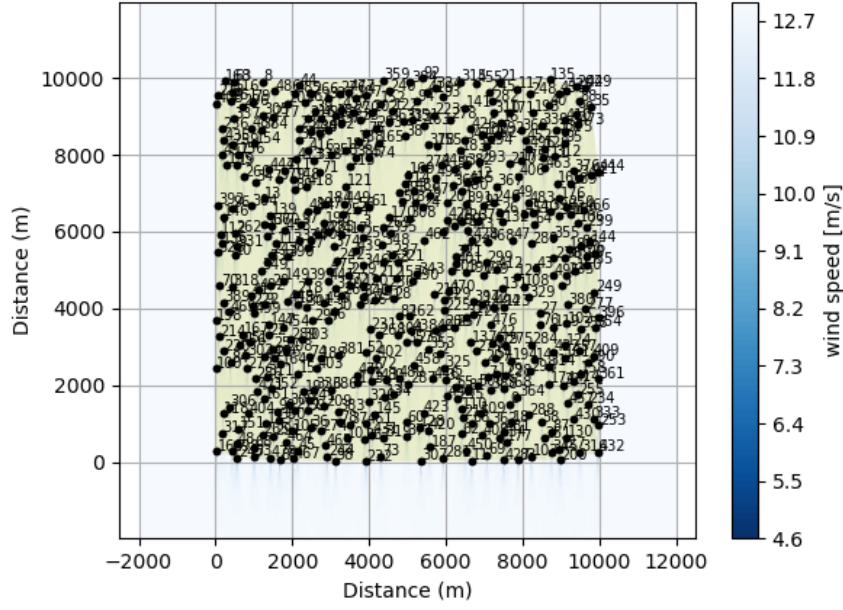
(b)



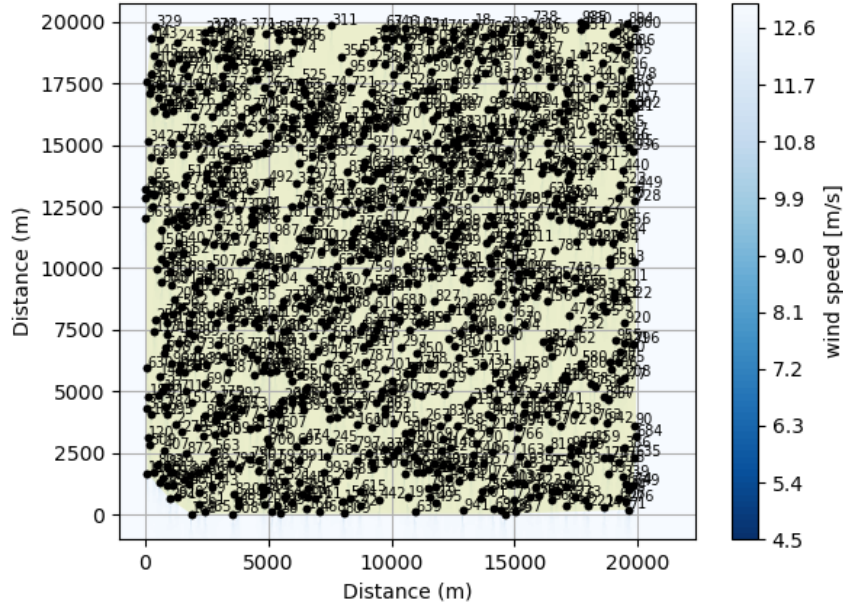
(c)



(d)



(e)



(f)

Figure 14: Layouts for increasing turbine numbers: (a) 30 turbines, (b) 100 turbines (3km grid size), (c) 100 turbines (5km grid size), (d) 100 turbines (10km grid size), (e) 500 turbines, (f) 1000 turbines.

The plots in Figure 14 are the flow map layouts, with the area of the farm highlighted in yellow. For 30 turbines, the layout is similar to that in Figure 13 as expected, and the turbines are fairly spread over the farm area. For the three results for 100 turbines (Figures 14b, 14c, 14d), we see that for the 3000m boundary, the turbines are positioned primarily in diagonal lines across the farm. As we increase the boundary lengths, we observed that the turbines are more spread out, with less uniform positioning. We note here that the algorithm would produce errors for 100 turbines on a 2000m grid, as PyWake would be unable to reposition turbines due to the 200m minimum separation distance, hence why we chose to start with 3000m for 100 turbines instead. For 500 turbines, we notice that the turbines mostly line the edge of the grid boundary, and the remainder of the turbines are spread over the farm (although there are some noticeable gaps). Finally, with 1000 turbines the layout appears to have “ridges” of turbine grouping, going across the farm. There are few gaps between these groups, although this may be due to the boundaries limiting the positioning.

Based on these results, we conclude that the grid boundary has a more significant impact on turbine positioning than initially anticipated. Expanding the size of the grid provides a larger search space for solutions, leading to more optimal outcomes. This could explain the decrease in uniformity observed in the 100-turbine cases. It also suggests that the results for the 500 and 1000 turbine cases are suboptimal, indicating that larger grid sizes should be considered in future calculations to improve these results.

Another point of improvement of this method could be the implementation of parallel programming. The DEAP library offers this in the form of the module Scalable Concurrent Operations in Python (SCOOP), which can distribute the optimisation across multiple processors [34]. This would reduce the runtime of the algorithm, enabling more generations to be calculated in the same amount of time, thus producing more optimal layouts.

Number of turbines	Grid side length (m)	AEP (GWh)	Area (km ²)	Runtime (s)
30	2000	133.36	3.54	121.78
100	3000	416.83	8.27	398.07
100	5000	439.78	22.34	377.94
100	10000	455.16	88.76	377.09
500	10000	2139.61	98.10	3715.47
1000	20000	4387.18	393.35	9898.61

Table 7: Results of scaling the number of turbines with NSGA-II.

Table 7 shows the values obtained from scaling the size of the wind farm. Naturally, as we increase the number of turbines, the power output and farm area increases. This increase is almost linear for the AEP, with 1000 turbines having a power output around thirty-three times greater than the layout for 30 turbines. For the area, we see that the farms are very close to the maximum area allowed by the boundaries; however, the farms are still smaller than that maximum, with the 1000 turbine layout having the largest difference in size (6.65km²).

With respect to supplying enough energy to power all homes in the UK, we infer that we would require approximately twelve farms with the layout presented in the 1000 turbine case (as each farm produces around 4.4TWh of energy annually). This amount may be reduced by increasing the grid boundaries in future calculations, to allow for better turbine spacing to reduce the wake effect. By direct comparison, we would require only five farms of the same scale as Hornsea 2 (165 turbines with farm size 462km², producing 11.6TWh annually [33]). However, this does not discredit our results; the turbines used in Hornsea 2 have a capacity of 8MW, with 81m blades reaching 200m above sea level at their tips [33]. These turbines are capable of generating significantly more energy than those characterised by Mosetti et al. and used in our calculations (3MW capacity, 20m blade length, 60m height), which helps explain the difference in the required number of farms [8].

Whilst the turbines used in our calculations differ in capacity and specifications from those in the Hornsea 2 project, our results still offer valuable insights into the usefulness of optimisation and NSGA-II in applications to the WFLOP. Future work could focus on adjusting the model to incorporate the advanced specifications of larger turbines to refine the results. While utilising the same turbines as those in Hornsea 2 would likely improve the results, it is important to recognise that since the farm came online in 2022, there have been further advancements in turbine technology. One such example is the Mingyang MySE18.X-20MW turbine, which was unveiled in 2023, featuring a 22MW capacity and maximum rated speed of 79.8m/s [35]. Additionally, exploring larger grid sizes in future simulations could further optimise turbine placement and improve efficiency.

6 Conclusion

In this report, we aimed to illustrate how useful single- and multi-objective optimisation can be in producing effective wind farm layouts. Utilising the characteristics presented by Mosetti et al., we explored the application of two different optimisation techniques - hill climbing and genetic algorithms - to design optimal wind farm layouts with the goal of scaling to meet the power demand of UK homes. Each method offered unique strengths in terms of effectiveness, accuracy and computational complexity. The HCA provided a straightforward approach to optimising individual turbine placement, but it proved too simplistic for use in the scaling scenario. NSGA-II, with its ability to handle multiple objectives simultaneously, demonstrated promising results in optimising power output and both cost and farm area, but at the cost of higher computational demands. Along with this, integrating PyWake into the algorithms allowed for a more precise application of the Jensen wake model in our calculations and facilitated the creation of more detailed layout plots of the results. Although it increased runtime, PyWake proved to be an effective tool in optimising wind farm layouts. Our results show that utilising an accurate wake model and a multi-objective evolutionary algorithm can produce effective layouts, even when constrained by computational resources.

Although the results obtained from the algorithms indicate that scaling wind farms to meet the growing power demands of the UK is feasible, further refinements are necessary. Improved turbine characteristics, expanding grid sizes, and applying more realistic wind and wake models could improve the results. Future work may consider other aspects of the WFLOP, such as environmental impact, logistical constraints, and integration with the existing power grid, to ensure that these wind farm layouts are not only optimal, but also practical for real-life implementation.

References

- [1] United States International Trade Administration. United Kingdom offshore wind, Aug 2022. Available at <https://www.trade.gov/market-intelligence/united-kingdom-offshore-wind>, Accessed 20-01-2025.
- [2] National Grid. The history of wind energy. Apr 2024. Available at <https://www.nationalgrid.com/stories/energy-explained/history-wind-energy#>, Accessed 20-01-2025.
- [3] Jordan Dearbail. Nine offshore wind farm projects awarded in UK auction, Sept 2024. Available at <https://www.bbc.co.uk/news/articles/c86ldzve4neo>, Accessed 20-01-2025.
- [4] Vanessa Martin. Chapter 5: Electricity. Technical report, Jul 2024. Available at <https://www.gov.uk/government/statistics/digest-of-uk-energy-statistics-dukes-2024>, Accessed 20-01-2025.
- [5] The Crown Estate. UK offshore wind report, 2023. Available at https://www.datocms-assets.com/136653/1720789954-11964_offshorewindreport_2023_final300424.pdf, Accessed 13-03-2023.
- [6] IBM Corporation. What is optimization modeling?, N. D. Available at <https://www.ibm.com/think/topics/optimization-model>, Accessed 23-01-2025.
- [7] Michele Samorani. The wind farm layout optimization problem. In *Handbook of wind power systems*, pages 21–38. Springer, 2014.
- [8] G. Mosetti, C. Poloni, and B. Diviacco. Optimization of wind turbine positioning in large windfarms by means of a genetic algorithm. *Journal of Wind Engineering and Industrial Aerodynamics*, 51(1):105–116, 1994.

- [9] Bukurije Hoxha, Alban Kuriqi, and Risto V Filkoski. Influence of seasonal air density fluctuations on wind speed distribution in complex terrains in the context of energy yield. *Energy, Ecology and Environment*, 9(2):175–187, 2024.
- [10] Jan Emblemsvåg. On the levelised cost of energy of windfarms. *International Journal of Sustainable Energy*, 39(7):700–718, 2020.
- [11] BVG Associates. Wind farm costs, 2019. Available at <https://guidetoanoffshorewindfarm.com/wind-farm-costs>, Accessed 25-02-2025.
- [12] Pawel L Manikowski, David J Walker, and Matthew J Craven. Multi-objective optimisation of the benchmark wind farm layout problem. *Journal of Marine Science and Engineering*, 9(12):1376, 2021.
- [13] Sanghyeon Jeon, Bumsuk Kim, and Jongchul Huh. Comparison and verification of wake models in an onshore wind farm considering single wake condition of the 2 MW wind turbine. *Energy*, 93:1769–1777, 2015.
- [14] F Azlan, JC Kurnia, BT Tan, and M-Z Ismadi. Review on optimisation methods of wind farm array under three classical wind condition problems. *Renewable and Sustainable Energy Reviews*, 135:110047, 2021.
- [15] I Katic, Jørgen Højstrup, and Niels Otto Jensen. A simple model for cluster efficiency. In *European wind energy association conference and exhibition*, pages 407–410. A. Raguzzi, 1987.
- [16] Linlin Tian, Weijun Zhu, Wenzhong Shen, Yilei Song, and Ning Zhao. Prediction of multi-wake problems using an improved Jensen wake model. *Renewable energy*, 102:457–469, 2017.
- [17] Xin-She Yang. *Nature-inspired optimization algorithms*. Elsevier, 1 edition, 2014.

- [18] Cuicui Lu, Hongyi Yuan, and Nianen Zhang. Nanophotonic devices based on optimization algorithms. In *Intelligent Nanotechnology*, pages 71–111. Elsevier, 2023.
- [19] John H Holland. *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.
- [20] Jason Brownlee. A gentle introduction to stochastic optimization algorithms, 2021. Available at <https://machinelearningmastery.com/stochastic-optimization-for-machine-learning/>, Accessed 12-02-2025.
- [21] Sittichoke Pookpant and Weerakorn Ongsakul. Optimal placement of wind turbines within wind farm using binary particle swarm optimization with time-varying acceleration coefficients. *Renewable energy*, 55:266–276, 2013.
- [22] SA Grady, M Yousuff Hussaini, and Makola M Abdullah. Placement of wind turbines using genetic algorithms. *Renewable energy*, 30(2):259–270, 2005.
- [23] Hüseyin Hakli. A new approach for wind turbine placement problem using modified differential evolution algorithm. *Turkish Journal of Electrical Engineering and Computer Sciences*, 27(6):4659–4672, 2019.
- [24] Russell Eberhart and James Kennedy. Particle swarm optimization. In *Proceedings of the IEEE international conference on neural networks*, volume 4, pages 1942–1948. Citeseer, 1995.
- [25] Xiuxing Yin, Xiaowei Zhao, Jin Lin, and Aris Karcianas. Reliability aware multi-objective predictive control for wind farm based on machine learning and heuristic optimizations. *Energy*, 202:117739, 2020.

- [26] Nidamarthi Srinivas and Kalyanmoy Deb. Multiobjective optimization using non-dominated sorting in genetic algorithms. *Evolutionary computation*, 2(3):221–248, 1994.
- [27] Weijie Zheng and Benjamin Doerr. Runtime analysis for the NSGA-II: Proving, quantifying, and explaining the inefficiency for many objectives. *IEEE Transactions on Evolutionary Computation*, 28(5):1442–1454, 2024.
- [28] Mads M. Pedersen, Alexander Meyer Forsting, Paul Van der Laan, Riccardo Riva, Leonardo A. Alcayaga Romàn, Javier Criado Risco, Mikkel Friis-Møller, Julian Quick, Jens Peter Schøler Christiansen, Rafael Valotta Rodrigues, Bjarke Tobias Olsen, and Pierre-Elouan Réthoré. PyWake 2.5.0: An open-source wind farm simulation tool. 2 2023.
- [29] Lie Meng Pang, Hisao Ishibuchi, and Ke Shang. NSGA-II with simple modification works well on a wide variety of many-objective problems. *IEEE Access*, 8:190240–190250, 2020.
- [30] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner, Marc Parizeau, and Christian Gagné. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13:2171–2175, Jul 2012.
- [31] Ryan Williams and Paul Domjan. The hidden costs of delivered renewable energy, Jul 2023. Available at <https://enodatech.com/news-insight/the-hidden-costs-of-delivered-renewable-energy>, Accessed 13-03-2025.
- [32] Angel G Gonzalez-Rodriguez, Javier Serrano-Gonzalez, Manuel Burgos-Payan, and Jesus Riquelme-Santos. Multi-objective optimization of a uniformly distributed offshore wind farm considering both economic factors and visual impact. *Sustainable Energy Technologies and Assessments*, 52:102148, 2022.
- [33] Orsted, Aug 2022. Available at <https://orsted.com/en/media/news/2022/08/20220831559011>, Accessed 15-03-2025.

- [34] Yannick Hold-Geoffroy, Olivier Gagnon, and Marc Parizeau. Once you SCOOP, no need to fork. In *Proceedings of the 2014 Annual Conference on Extreme Science and Engineering Discovery Environment*, page 60. ACM, 2014.
- [35] Mingyang Smart Energy Group. Wind Turbine Index. Available via <https://en.myse.com.cn/wind-turbine/index.aspx>, Accessed 24-03-2025.